
گزارش تشریح کد
استاد: جناب آقای دکتر پورفرد
تدریسار: جناب آقای دکتر قنبرپور
زهره حیدری و مهرداد احمدی
پاییز ۱۴۰۲
دانشگاه صنعتی امیرکبیر (پلی تکنیک تهران)

#include <stdio.h>

استفاده از توابع (scanf(),printf(),fopen(),fclose(),fprintf(),fscanf())

#include <SDL۲/SDL.h>

#include <SDL۲/SDL_ttf.h>

ساختن گزینه های رنگی برنامه

#include <stdlib.h>

استفاده از توابع (malloc(),free(),rand())

#include <string.h>

استفاده از توابع (strcmp(),strcpy(),strlen())

#include <ctype.h>

استفاده از تابع (tolower())

#include <time.h>

استفاده از تابع (clock())

متغیر صحیح numStudents تعداد دانشجویان را در هر مرحله نشان میدهد. این متغیر به صورت عمومی تعریف شده زیرا با توابع متفاوت برنامه تغییر میکند.

رشته ی عددی delete برای تابع deletefunc تعریف شده و هم در این تابع و در تابع search تغییر میکند. در واقع در تابع search شماره ردیف آن ردیفی که با نتایج سرچ مطابقت داشته به عنوان یک داده در این رشته ذخیره میشود سپس با فراخوانی تابع deletefunc این ردیف ها از لیست پیوندی حذف خواهند شد. چون این رشته در دو تابع کاربرد دارد و تغییر می یابد به صورت عمومی تعریف شده است.

متغیر صحیح count تعداد ردیف هایی که باید حذف شوند را نشان میدهد و در تابع search با پیدا شدن هر مطابقت مقدار آن افزایش می یابد سپس با استفاده از آن میتوانیم مشخص کنیم دقیقا چند سطر باید با استفاده از deletefunc از لیست پیوندی حذف شوند.

متغیر صحیح running نشان می دهد که برنامه باید به کار کردن ادامه دهد یا خیر. هنگامی که این متغیر مقدار یک داشته باشد برنامه اجرا خواهد شد ولی هنگامی که مقدار آن صفر شود برنامه نیز متوقف خواهد شد. مقدار آن توسط توابع متفاوت تغییر میکند بنابراین به صورت عمومی تعریف شده است.

Struct data:

یک ساختار به نام data و همچنین برچسب data ساخته شده است که اطلاعات هر دانشجو را نشان میدهد. این ساختار شامل این مقادیر است:

- یک اشاره گر از نوع همین ساختار به نام next : برای ساختن لیست پیوندی دوطرفه کاربرد دارد و به node بعدی اشاره میکند.
- اشاره گر از نوع همین ساختار به نام prev : برای ساختن لیست پیوندی دوطرفه کاربرد دارد و به node قبلی اشاره میکند.
- رشته ی حرفی firstName : سائز آن ۲۰ است (حداکثر میتواند ۱۹ کاراکتر را در خود ذخیره کند) و نشان دهنده ی نام دانشجو است.
- رشته ی حرفی lastName : سائز آن ۲۰ است (حداکثر میتواند ۱۹ کاراکتر را در خود ذخیره کند) و نشان دهنده نام خانوادگی دانشجو است.
- رشته ی حرفی ID : نشان دهنده ی کد دانشجویی است.
- رشته ی حرفی city : نشان دهنده ی نام شهر دانشجو است.

نکته ی قابل توجه آن است که سعی شده متغیرهای درون این ساختار باتوجه به padding چیده شوند تا کمترین فضا به آنها اختصاص داده شود.

lowercase():

این تابع یک رشته ی حرفی را میگیرد و با استفاده از تابع tolower() تمام حروف آن را به حروف کوچک تبدیل میکند. از آنجایی که برنامه نباید به بزرگ و کوچک بودن حروف حساس باشد از این تابع تقریباً در تمام توابع دیگر استفاده میشود.

creatStudents():

این تابع یک داده از نوع data ایجاد میکند که شامل اطلاعات یک دانشجو است. ورودی های این تابع شامل نام، نام خانوادگی، شماره دانشجویی و شهر محل زندگی دانشجو می باشد. این تابع این اطلاعات را در یک ساختار داده ای ذخیره می کند و یک اشاره گر به آن ساختار را برمی گرداند.

از این تابع برای ساختن لیست پیوندی (در قسمت خواندن فایل و همچنین در قسمت مرتب سازی برای ساختن لیست پیوندی جدید) استفاده می شود. البته این تابع فقط اطلاعات گرفته شده را به صورت ساختار ذخیره میکند ولی دو متغیر next, prev را NULL قرار می دهد. و این متغیرها در صورت نیاز به صورت دیگری تغییر می کنند.

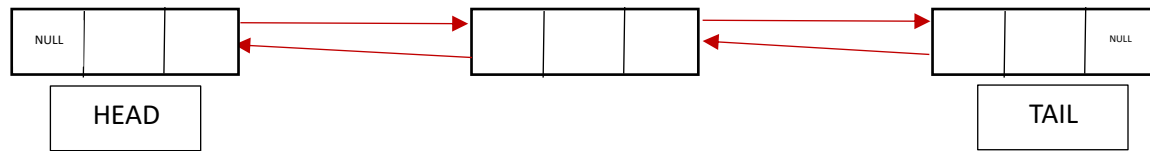
ReadFile():

این تابع یک فایل متنی به نام "students.txt" را می خواند (این فایل متنی در همان پوشه ی برنامه ذخیره شده است و در صورتی که قادر به باز کردن فایل مورد نظر نباشد با ارور مربوطه کاربر را آگاه میکند.) و اطلاعات دانشجویان را از آن استخراج می کند. (این اطلاعات به این صورت وارد شده اند که ۲۰ کاراکتر اول مربوط به نام دانشجو، ۲۰ کاراکتر بعد مربوط به نام خانوادگی او، ۲۰ کاراکتر بعد مربوط به کد دانشجویی و در انتها ۲۰ کاراکتر مربوط به شهر محل سکونت وی است و اطلاعات هر دانشجو در یک سطر نوشته شده است) سپس این اطلاعات را قابل استفاده میکند یعنی هم آن ها را به حروف کوچک تبدیل میکند و همچنین چک میکند که ID (کد های داوطلبی) دانشجویان متفاوت باشد. اگر متفاوت نباشد با اروری کاربر را آگاه میکند و آن دو کد داوطلبی را در صفحه نمایش می دهد و مقدار ID_diff را صفر می کند که با این کار برنامه متوقف خواهد شد. اگر کدهای داوطلبی متفاوت بودند این اطلاعات را به یک لیست پیوندی دو طرفه و یک آرایه از اشاره گرها به نام students ذخیره می کند.

به عنوان ورودی، این تابع سه اشاره گر به داده از نوع "data" به نام head, tail و students[] و یک اشاره گر به عنوان ID_diff را دریافت می کند. در نهایت، این تابع یک اشاره گر به ساختار داده ای "data" به نام head را برمی گرداند.

برای ساخت لیست پیوندی دو طرفه به تعداد numStudents تابع createStudent فراخوانی میشود و اطلاعات از فایل خوانده میشود و سپس با استفاده از این تابع دانشجو ساخته میشود. سپس با استفاده از head, tail این دانشجویان به

یکدیگر لینک میشوند. به این صورت که در یک لیست پیوندی دو طرفه tail به آخرین دانشجوی اضافه شده اشاره می کند و head اولین دانشجوی اضافه شده است.



ListToArray():

هنگام اجرای برنامه گاهی اوقات تغییرات بر روی لیست پیوندی انجام میشوند و نیاز است آرایه ی students هم آپدیت شود (به عنوان مثال در هنگام اضافه کردن دانشجو یا حذف کردن دانشجو). این تابع یک اشاره گر از نوع ساختار data و همچنین آرایه ی students را به عنوان ورودی دریافت می کند و یک اشاره گر از نوع data به نام current ایجاد میکند و آن را برابر head قرار میدهد سپس با استفاده از next روی لیست پیوندی پیمایش میکند و اطلاعات آن را در آرایه ذخیره میکند. این تابع از نوع void است.

ArrayToList():

همچنین هنگام اجرای برنامه گاهی اوقات تغییرات روی آرایه انجام می شوند و نیاز است لیست پیوندی آپدیت شود (به عنوان مثال در هنگام مرتب سازی). این تابع یک اشاره گر از نوع ساختار data و همچنین آرایه ی students را به عنوان ورودی دریافت می کند و دو ساختار از نوع data با نام های head^۲ و tail^۲ می سازد. سپس با استفاده از تابع createStudent و الگوریتم توضیح داده شده در قسمت تابع read یک لیست پیوندی دو طرفه ی جدید با اطلاعات موجود در آرایه و پیمایش روی آرایه میسازد و سپس head^۲ که اشاره گری به اولین node این لیست پیوندی دو طرفه می باشد را برمیگرداند.

توجه کنید که در این حالت باید یک لیست کاملاً جدید ایجاد گردد زیرا تمام لینک های لیست پیوندی قبلی غیر استفاده خواهند بود و تمام آنها باید از اول به وجود آیند.

Insert():

این تابع برای اضافه کردن دانشجو با استفاده از لیست پیوندی کاربرد دارد.

ورودی های تابع آرایه students و اشاره گری از نوع data به نام head است که در واقع به خانه ی اول لیست پیوندی فعلی اشاره می کند و همچنین اشاره گری به مقدار ID_diff است، که نشان می دهد کد های داوطلبی دانشجویان متفاوت است یا خیر، هستند.

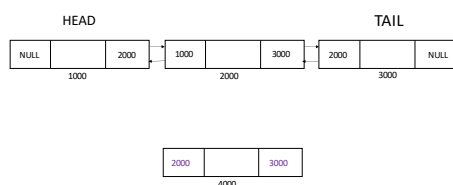
در ابتدا اطلاعات دانشجوی مورد نظر از کاربر پرسیده میشوند و معتبر بودن آن اطلاعات بررسی می شوند. به طور مثال اگر کد داوطلبی داده شده با یک کد داوطلبی دانشجویی که قبلا ذخیره شده است یکسان باشد به کاربر پیام خطا میدهد و اطلاعات آن دانشجو را به کاربر نشان میدهد. همچنین مقدار ID_diff* صفر میشود که با استفاده از آن running صفر شده و برنامه متوقف خواهد شد.

همچنین واضح است ردیفی که کاربر اعلام میکند که دانشجوی جدید در آن اضافه شود باید مقداری بین یک و تعداد دانشجویان باشد. بنابراین اگر آن مقدار کمتر از یک باشد برنامه به صورت خودکار آن را به یک تغییر میدهد و دانشجو را در ردیف اول اضافه می کند و اگر مقدار آن از تعداد دانشجویان بیشتر باشد برنامه به صورت خودکار دانشجوی جدید را در انتهای لیست اضافه می کند.

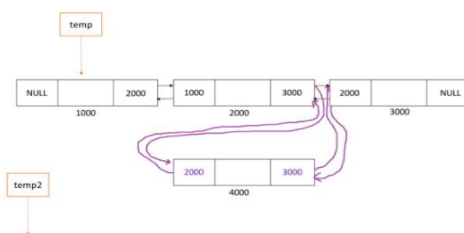
اضافه شدن دانشجو به لیست به این صورت انجام می شود که در ابتدا یک اشاره گر از نوع data به نام newStudent ایجاد میشود و با استفاده از تابع createStudent اطلاعات داده شده در آن ذخیره می گردد.

سپس دو اشاره گر از نوع data به نام های temp_prev و temp2_next ایجاد میشوند. اشاره گر temp_prev باید به node قبل از دانشجوی جدید و temp2_next باید به node بعد از دانشجوی جدید اشاره کند.

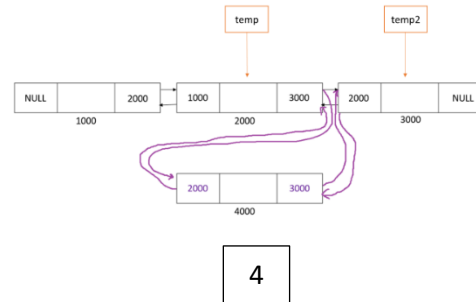
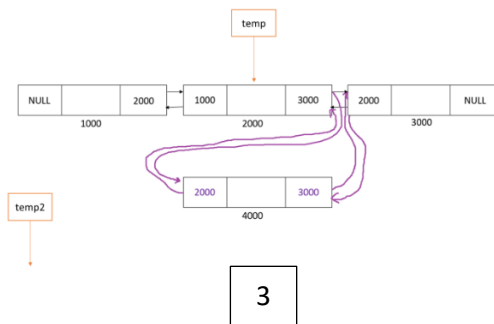
به منظور اجرای این کار ابتدا temp_prev را برابر head که در واقع اولین خانه ی لیست پیوندی دو طرفه ی ماست قرار می دهیم. سپس با استفاده از حلقه و شماره ردیفی که از کاربر دریافت کرده ایم و همچنین با دستور temp_prev = temp_prev -> next روی لیست پیوندی پیمایش میکنیم تا به خانه ی مورد نظر برسیم. سپس خانه ی بعدی آن را برابر temp2_next قرار می دهیم. اکنون باید بین این دو خانه یک خانه اضافه کنیم و اطلاعات دانشجوی جدید را در آن قرار دهیم و لینک های لیست را آپدیت کنیم. این کار را به این صورت انجام می دهیم که next در temp_prev را همان newStudent قرار میدهیم (خانه ی بعد از خانه مورد نظر را دانشجوی جدید قرار می دهیم). سپس prev در temp2_next را همان newStudent قرار میدهیم (خانه ی قبل از خانه ی مورد نظر را دانشجوی جدید قرار می دهیم). و سپس next و prev آن دانشجوی جدید را آپدیت میکنیم.



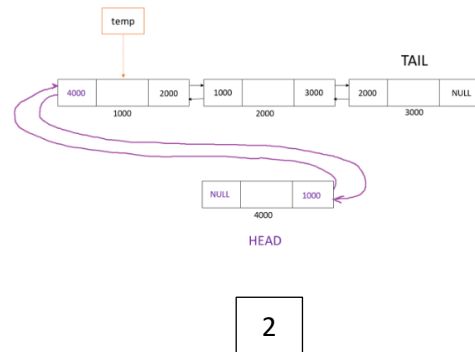
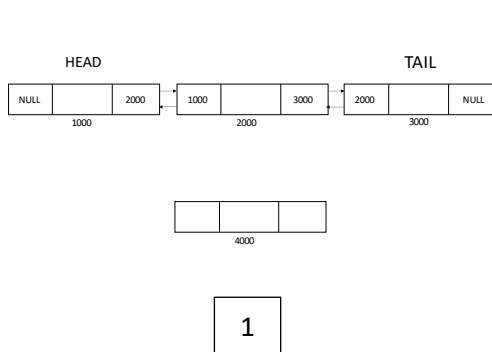
1



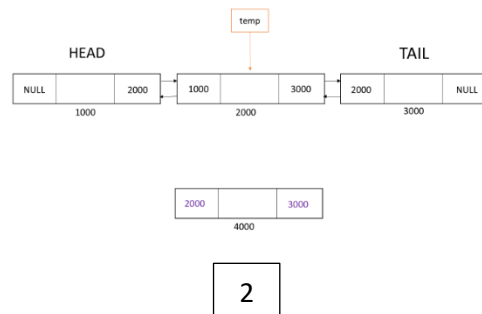
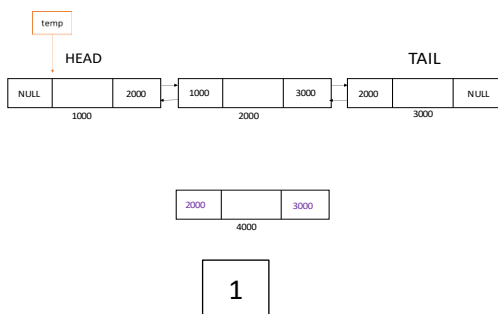
2

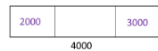
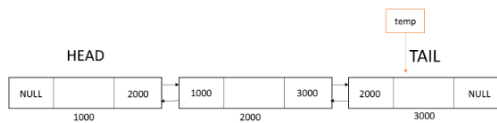


البته در صورتی که کاربر بخواهد دانشجوی مورد نظر را در ابتدا یا انتهای لیست اضافه کند، الگوریتم اضافه کردن کمی تغییر می کند. به این صورت که اگر کاربر در ابتدا عدد یک را وارد کرده باشد (که برنامه یکی کمتر از آن یعنی صفر را در نظر میگیرد چون برای پیمایش و فهمیدن خانه ای که به خانه ی قبل از دانشجوی جدید تبدیل می شود به آن احتیاج دارد) این دانشجوی جدید به head جدید باید تبدیل شود. در این راستا ابتدا prev آن مقدار NULL می شود و next آن به head فعلی اشاره می کند. سپس head->prev فعلی به همین دانشجوی جدید اشاره می کند. و در آخر head به دانشجوی جدید اشاره می کند. به این صورت دانشجو به ابتدای لیست اضافه می شود.

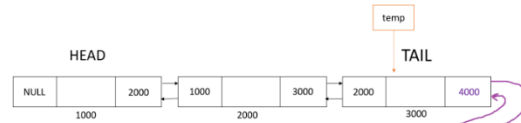


در صورتی که کاربر بخواهد دانشجوی مورد نظر در انتهای لیست اضافه کند، با پیمایش انجام شده روی لیست پیوندی قبلی به خانه ای می رسد که next آن مقدار NULL را دارد. (همان خانه tail). بنابراین با رسیدن به این مقدار برنامه میفهمد باید دانشجو را به انتهای لیست اضافه کند. پس next این مقدار temp_prev را دانشجوی جدید قرار میدهد. Prev دانشجوی جدید را temp_prev قرار می دهد و در انتها next دانشجوی جدید را NULL می کند.





3



4

به این صورت دانشجوی مورد نظر به لیست پیوندی اضافه می شود ولی هنوز آرایه آپدیت نشده است. بنابراین در ابتدا تعداد دانشجویان (numStudents) را یکی افزایش می دهیم. سپس آرایه را با استفاده از تابع ListToArray() آپدیت می کنیم.

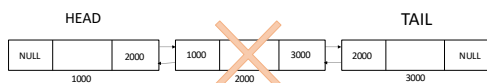
سپس یک اشاره گر از نوع data را باز می گردانند. که اشاره گری به خانه ی اول لیست پیوندی است.

Deletefunc():

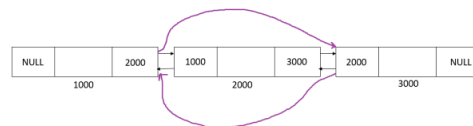
برای حذف کردن دانشجویی که با اطلاعات داده شده از کاربر مطابقت دارد.

برای این تابع نیز ورودی ها همان آرایه students و اشاره گری از نوع data به نام head است که در واقع به خانه ی اول لیست پیوندی فعلی اشاره می کند و همچنین مقدار عددی ردیفی که باید حذف شود. تابع دو اشاره گر از نوع data به نام های temp_del و temp_2 می سازد. temp_del باید به خانه ای از لیست پیوندی که باید حذف شود اشاره کند و temp_2 خانه ی بعدی آن خواهد بود. برای این منظور ابتدا با استفاده از عدد ردیفی که از کاربر دریافت شده است و همچنین یک حلقه روی لیست پیوندی پیمایش میشود تا temp_del به همان خانه ای که باید حذف شود اشاره کند.

سپس temp_2->prev باید به خانه ی قبل خانه ی مورد نظر برای حذف اشاره کند. سپس next برای خانه ی قبل از خانه ی مورد نظر برای حذف هم باید به آن خانه ی بعدی یعنی (temp_2) اشاره کند. سپس خانه ی مورد نظر می تواند حذف شود.



1



2

در این تابع نیز حالات خاص ابتدا و انتهای لیست در نظر گرفته شده اند. اگر خانه ای که باید حذف شود در ابتدای لیست باشد تنها باید prev خانه ی بعدی آن را NULL کنیم یعنی آن خانه ی بعدی به head جدید تبدیل شود.



1

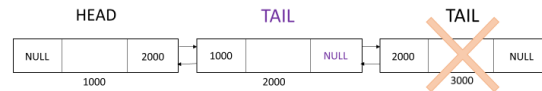


2

اگر خانه ای که باید حذف شود در انتهای لیست باشد، prev خانه ی آخر را temp₂ میکنیم سپس temp₂->next را NULL قرار می دهیم به این صورت این خانه به tail تبدیل خواهد شد.



1



2

این تابع لیست پیوندی را آپدیت می کند. بنابراین باز هم باید تعداد دانشجویان (numStudents) را یکی کاهش دهیم و سپس با استفاده از فراخوانی تابع ListToArray() آرایه را نیز آپدیت نماییم. خروجی این تابع یک اشاره گر از نوع data است که در واقع خانه ی ابتدای لیست پیوندی است.

writeToFile():

این تابع برای نوشتن اطلاعات روی فایل با نام "student.txt" (همان فایل اصلی) و همچنین فایل باینری "output.bin" استفاده میشود. به این صورت که با مود نوشتن این فایل متنی را باز میکند و در صورتی که نتواند پیام خطای متناسب را به کاربر اعلام می کند. سپس اشاره گری از نوع data می سازد. که به خانه ای که اطلاعات آن در حال نوشته

شدن روی فایل اند را اشاره می کند. در ابتدا این اشاره گر را برابر head که خانه ی اول لیست پیوندی است قرار می دهد. سپس اطلاعات را با استفاده از تابع fprintf() به صورت خواسته شده (که برای هر قسمت ۲۰ کاراکتر در نظر گرفته شود) و پیمایش روی لیست پیوندی در فایل متنی می نویسد. و فایل را می بندد.

سپس دوباره تمام این مراحل را با باز کردن (و یا ایجاد کردن) فایل بایزری آغاز کرده و اطلاعات را بر روی آن ذخیره می کند. سپس آن فایل را نیز می بندد.

Print():

در بسیاری از توابع پس از اینکه کاربر تغییرات مورد نیاز را ایجاد کرد، نیاز است که لیست کامل تغییر یافته به کاربر نشان داده شود بنابراین برای جلوگیری از تکرار این تابع تعریف شده است.

این تابع یک اشاره گر از نوع data را به عنوان ورودی دریافت می کند که اشاره گری به خانه ی اول لیست پیوندی است. سپس یک اشاره گر از نوع data به نام current میسازد که به خانه ای اشاره می کند که اطلاعات آن در حال پرینت شدن روی صفحه می باشند. سپس روی لیست پیوندی پیمایش می کند (تا جایی که مقدار آن NULL نشود) و آن اطلاعات را به صورت خواسته شده یعنی به شمایل یک جدول (که برای هر قسمت ۲۰ کاراکتر در نظر گرفته می شود) پرینت می کند. سپس عناوین مناسب را در ابتدا پرینت می کند.

خواندن فایل:

با کلیک کردن کاربر روی قسمت مربوط به خواندن فایل (rect۱) تابع readFile فراخوانی میشود که اطلاعات را از فایل مورد نظر میخواند. همچنین مقدار flag_read برابر یک می شود که برای این استفاده می شود که اگر کاربر قبل از آنکه روی این گزینه کلیک کند، گزینه ی دیگری را انتخاب نماید، خطای متناسب به او نشان داده شود. سپس با فراخوانی تابع print() اطلاعات در صفحه نمایش داده می شوند.

بخش جست و جوی دانشجو:

با انتخاب گزینه ی «search» از منوی رنگی، تابع «search» فراخوانی می شود. این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی، اشاره گر به آرایه دانشجویان در ساختار «data»، آرایه ای از اعداد صحیح به نام «delete» و اشاره گر به متغیر صحیح «count» را به عنوان ورودی دریافت می کند و در نهایت اطلاعات دانشجوی مورد نظر کاربر را نمایش می

دهد. (توجه شود که در بخش جست و جو ما با دو آرگومان آرایه ای از اعداد صحیح و اشاره گر به متغیر صحیح کاری نداریم و در بخش « حذف دانشجو » از آنها استفاده می شود). با فراخوانی این تابع ابتدا از کاربر خواسته می شود عنوان ستون مورد نظر خود را وارد کند و از او خواسته می شود که حتما یکی از عنوان های «row»، «first_name»، «Last_name»، «ID» یا «city» را حتما به همان شکل خواسته شده وارد نماید. سپس با توجه به عنوان انتخاب شده از کاربر خواسته می شود که عبارت مد نظر خود (ردیف ، نام ، نام خانوادگی ، شماره دانشجویی یا نام شهر) را وارد نماید. بعد از این مراحل سطر اطلاعات دانشجوی مورد نظر کاربر همراه با شماره ی سطر در فایل اصلی ، در ترمینال نمایش داده می شود. توجه شود که اگر عبارت مدنظر کاربر بیشتر از یکی درون فایل اصلی موجود باشد ، اطلاعات تمام دانشجویان دارای آن عنوان در سطرهای مختلف نمایش داده می شود. همچنین اگر عبارت مدنظر کاربر در فایل اصلی موجود نباشد ، با عبارتی مبنی بر «عبارت مورد نظر یافت نشد مواجه می شود». توجه شود که در هیچ کدام از مراحل «وارد کردن عنوان» و «وارد کردن عبارت مورد نظر» برنامه به حروف بزرگ و کوچک حساس نمی باشد و در هر کدام از این مراحل برنامه با فراخوانی تابع «lower case» این حساسیت از بین رفته است. لازم به ذکر است که تمامی مراحل تابع «search» مستقیم بر روی لیست پیوندی انجام می شود.

بخش بهم ریختن تصادفی اطلاعات دانشجویان:

با انتخاب گزینه ی «random» از منوی رنگی ، ابتدا از کاربر خواسته می شود در صورت تمایل به بهم ریختن تصادفی اطلاعات دانشجویان عبارت «random_changing» را به همین صورت وارد نماید و در صورت وارد شدن درست این عبارت تابع «random_changing» فراخوانی می شود. این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی ، اشاره گر به آرایه ی دانشجویان در ساختار «data» و تعداد دانشجویان را به عنوان ورودی دریافت می کند. با فراخوانی این تابع اطلاعات دانشجویان به صورت تصادفی بهم می خورد و در لیست پیوندی ذخیره می شود و در نهایت نیز این تابع ، لیست پیوندی را از ابتدای آن بر می گرداند. بعد از آن نیز با فراخوانی تابع «print» اطلاعات دانشجویان که به صورت تصادفی بر هم خورده است ، نمایش داده می شود. توجه شود که در مرحله درخواست از کاربر برای وارد کردن عبارت «random_changing» برنامه به وارد کردن حروف بزرگ و کوچک حساس نیست و با فراخوانی تابع «lowercase» در این مرحله ، این حساسیت از بین رفته است.

بخش مرتب سازی اطلاعات دانشجویان:

با انتخاب گزینه ی «sorting» از منوی رنگی ، ابتدا از کاربر خواسته می شود عنوانی که مرتب سازی بر اساس آن مد نظر است ، وارد شود. این عناوین باید حتما به یکی از صورت های «first_name»، «last_name»، «ID» و «city» وارد شود.

سپس از کاربر خواسته می شود یکی از روش های «insertion_sort»، «selection_sort»، «bubble_sort» و «quick_sort» وارد شود. توجه شود که روش مورد نظر حتما باید به همین شکل ها وارد شود و در صورت وارد کردن شکلی دیگر، عملیات مرتب سازی انجام نمی شود. بعد از این مرحله، از کاربر خواسته می شود نوع مرتب سازی (صعودی یا نزولی) را وارد کند. در این مرحله کاربر حتما باید یکی از عبارت های «upward» و «downward» را حتما به همین شکل های نوشته شده وارد نماید. سپس با توجه به عناوین انتخابی کاربر، یکی از توابع «insertion_sort»، «selection_sort»، «bubble_sort» و «quick_sort» فراخوانی می شود که درباره ی هر یک به طور جداگانه توضیح خواهیم داد. همچنین با استفاده از دستور «clock» زمان مرتب سازی تابع مورد نظر در متغیری به نام «time_taken» ذخیره می شود. در نهایت نیز لیست مرتب شده ی دانشجویان به همراه روش انتخابی مرتب سازی، نوع مرتب سازی (صعودی یا نزولی بودن)، عنوانی که مرتب سازی بر اساس آن صورت گرفته است و زمان صرف شده برای مرتب سازی، نمایش داده می شود. توجه شود که در هیچ کدام از مراحل «انتخاب عنوان»، «انتخاب روش مرتب سازی» و «انتخاب نوع مرتب سازی»، برنامه به حروف بزرگ و کوچک حساس نمی باشد و در هر کدام از این مراحل با فراخوانی تابع «lowercase» این حساسیت از بین رفته است.

تابع insertion_sort

این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی، آرایه ی دانشجویان در ساختار «data»، تعداد دانشجویان، متغیر flag و متغیر x را به عنوان ورودی دریافت می کند. سپس با توجه به مقدار x که هنگام انتخاب عنوانی که مرتب سازی بر اساس آن انجام می شود و همچنین مقدار flag که نوع مرتب سازی بر اساس آن انجام می شود، اطلاعات دانشجویان را مرتب و در لیست پیوندی ذخیره می کند و در نهایت نیز لیست پیوندی را از ابتدای آن بر می گرداند. اگر $x=1$ ، مرتب سازی بر اساس نام دانشجویان و حروف الفبا انجام می شود. اگر $x=2$ ، مرتب سازی بر اساس نام خانوادگی دانشجویان و حروف الفبا انجام می شود. اگر $x=3$ ، مرتب سازی بر اساس شماره دانشجویی دانشجویان و اعداد انجام می شود و در نهایت اگر x در طول برنامه مقدار دهی نشده باشد، مرتب سازی بر اساس نام شهر و حروف الفبا انجام می شود. در هر کدام از این مرتب سازی ها، اگر $flag=1$ مرتب سازی صعودی و اگر $flag=0$ مرتب سازی نزولی انجام می شود.

تابع selection_sort

این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی، آرایه ی دانشجویان در ساختار «data»، تعداد دانشجویان، متغیر flag و متغیر x را به عنوان ورودی دریافت می کند. سپس با توجه به مقدار x که هنگام انتخاب عنوانی که مرتب سازی بر اساس آن انجام می شود و همچنین مقدار flag که نوع مرتب سازی بر اساس آن انجام می شود، اطلاعات دانشجویان را مرتب و در لیست پیوندی ذخیره می کند و در نهایت نیز لیست پیوندی را از ابتدای آن بر می گرداند. اگر $x=1$ ، مرتب سازی بر اساس نام دانشجویان و حروف الفبا انجام می شود. اگر $x=2$ ، مرتب سازی بر اساس نام خانوادگی دانشجویان و حروف الفبا انجام می شود. اگر $x=3$ ، مرتب سازی بر اساس شماره دانشجویی دانشجویان و اعداد انجام می

شود و در نهایت اگر x در طول برنامه مقدار دهی نشده باشد، مرتب سازی بر اساس نام شهر و حروف الفبا انجام می شود. در هر کدام از این مرتب سازی ها، اگر $flag=1$ مرتب سازی صعودی و اگر $flag=0$ مرتب سازی نزولی انجام می شود.

تابع bubble_sort

این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی، آرایه ی دانشجویان در ساختار «data»، تعداد دانشجویان، متغیر $flag$ و متغیر x را به عنوان ورودی دریافت می کند. سپس با توجه به مقدار x که هنگام انتخاب عنوانی که مرتب سازی بر اساس آن انجام می شود و همچنین مقدار $flag$ که نوع مرتب سازی بر اساس آن انجام می شود، اطلاعات دانشجویان را مرتب و در لیست پیوندی ذخیره می کند و در نهایت نیز لیست پیوندی را از ابتدای آن بر می گرداند. اگر $x=1$ ، مرتب سازی بر اساس نام دانشجویان و حروف الفبا انجام می شود. اگر $x=2$ ، مرتب سازی بر اساس نام خانوادگی دانشجویان و حروف الفبا انجام می شود. اگر $x=3$ ، مرتب سازی بر اساس شماره دانشجویی دانشجویان و اعداد انجام می شود و در نهایت اگر x در طول برنامه مقدار دهی نشده باشد، مرتب سازی بر اساس نام شهر و حروف الفبا انجام می شود. در هر کدام از این مرتب سازی ها، اگر $flag=1$ مرتب سازی صعودی و اگر $flag=0$ مرتب سازی نزولی انجام می شود.

تابع quick_sort

این تابع آرگومان های اشاره گر به ابتدای لیست پیوندی، آرایه ی دانشجویان در ساختار «data»، عدد صفر، تعداد یکی کمتر از تعداد دانشجویان، متغیر $flag$ و متغیر x را به عنوان ورودی دریافت می کند. سپس با توجه به مقدار x که هنگام انتخاب عنوانی که مرتب سازی بر اساس آن انجام می شود و همچنین مقدار $flag$ که نوع مرتب سازی بر اساس آن انجام می شود، اطلاعات دانشجویان را مرتب و در لیست پیوندی ذخیره می کند و در نهایت نیز لیست پیوندی را از ابتدای آن بر می گرداند. اگر $x=1$ ، مرتب سازی بر اساس نام دانشجویان و حروف الفبا انجام می شود. اگر $x=2$ ، مرتب سازی بر اساس نام خانوادگی دانشجویان و حروف الفبا انجام می شود. اگر $x=3$ ، مرتب سازی بر اساس شماره دانشجویی دانشجویان و اعداد انجام می شود و در نهایت اگر x در طول برنامه مقدار دهی نشده باشد، مرتب سازی بر اساس نام شهر و حروف الفبا انجام می شود. در هر کدام از این مرتب سازی ها، اگر $flag=1$ مرتب سازی صعودی و اگر $flag=0$ مرتب سازی نزولی انجام می شود.

اضافه کردن دانشجو:

با کلیک کردن کاربر بر گزینه ی اضافه کردن دانشجو (rect3) در ابتدا چک میشود که قبلا کاربر بر روی گزینه ی خواندن فایل کلیک کرده باشد، سپس تابع insert فراخوانی می شود و دانشجوی مورد نظر به لیست اضافه میشود. در صورتی که

دانشجو معتبر باشد و کد دانشجویی متفاوتی با بقیه داشته باشد، اطلاعات جدید با استفاده از print() بر روی صفحه نمایش می یابند.

حذف دانشجو:

با کلیک کردن کاربر بر گزینه ی حذف دانشجو(rect4) در ابتدا چک میشود که قبلا کاربر بر روی گزینه ی خواندن فایل کلیک کرده باشد، سپس تابع search فراخوانی می شود و ستون مورد نظر و کلمه ی مورد نظر از کاربر خواسته می شود. و اطلاعات ردیف های مطابقت داشته در رشته ی delete ذخیره می شوند و مقدار count هم به صورت متناسب آپدیت می شود. سپس با فراخوانی تابع deletefunc() و یک حلقه هربار یک دانشجو از لیست حذف میشود. سپس هنگامی که تمام دانشجویان مطابقت داشته از لیست حذف شدند، اطلاعات تغییر یافته با فراخوانی تابع print() در صفحه نمایش داده می شوند.

نوشتن فایل:

با کلیک کردن کاربر بر گزینه ی نوشتن فایل (rectV) در ابتدا چک میشود که قبلا کاربر بر روی گزینه ی خواندن فایل کلیک کرده باشد، سپس تابع writeToFile فراخوانی می شود و به این صورت اطلاعات روی فایل نوشته می شوند. البته تمام flag های استفاده شده برای نشان دادن استفاده شدن گزینه های قبل بدون استفاده از نوشتن تابع، صفر می شوند. این مقدارها برای آن قرار داده شده اند که اگر کاربر تغییری در لیست ایجاد نماید ولی بر روی گزینه ی نوشتن فایل کلیک نکند و بخواهد برنامه را ببندد برنامه هشدار مناسبی چاپ کند و از اون بپرسد بابت اینکار مطمئن است یا خیر. بنابراین وقتی کاربر بر روی نوشتن فایل کلیک می کند باید این مقادیر صفر شوند تا در صورت نیاز دوباره به یک تبدیل شوند.

بستن فایل:

با کلیک کردن کاربر بر بستن فایل (rect8) در ابتدا چک میشود اگر کاربر بر روی هر یک از گزینه های اضافه کردن دانشجو، حذف کردن دانشجو، مرتب سازی، جست و جو ، یا بهم زدن تصادفی کلیک کرده باشد بدون اینکه اطلاعات را در فایل نوشته باشد، هشدار مناسبی چاپ کند و از اون بپرسد بابت اینکار مطمئن است یا خیر. در صورتی که کاربر yes (بزرگ بودن و کوچک بودن حروف اهمیتی ندارد) را وارد کند برنامه بسته خواهد شد و در صورتی که no (بزرگ بودن و کوچک بودن حروف اهمیتی ندارد) وارد شود، برنامه اجازه میدهد کاربر گزینه های دیگری انتخاب نماید.

در آخرفضای برنامه ی گرافیکی و همچنین فضای اختصاص داده شده با استفاده از تابع free() و پیمایش بر روی لیست پیوندی با شروع از head خالی می گردد.

بخش SDL:

برای نوشتن کد مربوط به رابط کاربری و جدول نمایش داده شده در ابتدای برنامه که شامل گزینه های مختلف می باشد از فیلم فرستاده شده توسط جناب آقای شفیع زاده استفاده شد.

```
//readfile
SDL_Rect rect1 = {10,10,500,90};
SDL_Rect text_rect_readFile= {10,10,500,90};
SDL_SetRenderDrawColor(rend,255,198,217,255);
SDL_RenderFillRect(rend,&rect1);
get_text_and_rect(rend, text_rect_readFile.x + 10, text_rect_readFile.y + 10,"read file!", font, &texture, &text_rect_readFile);
SDL_RenderCopy(rend,texture,NULL,&text_rect_readFile);
```

در قسمت هایی که به صورت کد بالا هستند، مستطیل های مرتبط با هر گزینه ساخته شده است. که سایز آنها ۹۰ و ۵۰۰ بوده است و اینکه کجای صفحه قرار گیرند نیز به صورت دستی به آنها داده شده است.

رنگ استفاده شده برای هر گزینه رنگ صفحه ی کلی (window) در فیلم می باشد. (همچنین رنگ window همان رنگ گزینه های استفاده شده در فیلم می باشد).

در کدهای مشابه کد بالا، شرط چک شده است که کاربر بر روی مستطیل مورد نظر (rect۱ در کد بالا) کلیک نموده است یا

```
if (SDL_PointInRect(&mousePosition,&rect1)) // read file
{
    flag_read = 1;
    printf("reading file...\n");
    head = readfile(head,tail,students,&ID_diff);
    // Print the student information from the doubly linked list
    if (ID_diff)
    {
        print(head);
    }
}
```

خیر و اگر کلیک کرده باشد کد های مربوطه اجرا خواهند شد.